

Tracelane: A Serverless Pipeline for Real-Time Freight Visibility with Rule-Aware ETA Prediction

David Fellner · TU Wien · Course project, TU Wien × Amazon Web Services, 2026 · Code: *see repository README*

Abstract. Road freight carriers often cannot tell customers where a shipment is or when it will arrive. I designed and implemented Tracelane, an end-to-end serverless system on AWS that ingests truck telemetry over MQTT, processes it with at-least-once semantics, and serves live tracking, predicted arrival times and service-level metrics to operators and their customers. Because real telematics data is proprietary, I built a fleet simulator on real European road geometry (37 hubs, 156 lanes, 13 countries) that models EU driving-time rules, border controls, traffic, refrigerated cargo and emissions, and serves as ground truth for evaluation. Over 5 seeds × 30 simulated days (11,139 deliveries), an ETA predictor that encodes driving-time rules and customs reduces the mean absolute error at 10 % route progress from 220 to 27 minutes compared with a distance/speed baseline (8.0×). A fault-injection experiment shows that the ingestion path converges to an identical state under duplicated, reordered and malformed messages; it also exposed an order-dependence bug, which I fixed. Limitations — above all, synthetic data — are discussed in Section 5.

1 Problem and goals

Visibility in road freight is still largely manual: dispatchers phone drivers and forward status by e-mail. The project brief (TU Wien × AWS) asked for a proof of concept that turns GPS events into a live dashboard. I extended the brief to four goals: **G1** low latency from GPS ping to screen (design target < 5 s); **G2** correct state under the at-least-once, possibly reordered delivery that MQTT and stream processing imply; **G3** information that is useful beyond a dot on a map — realistic arrival predictions and service metrics such as on-time rate and CO₂e; **G4** no servers to operate and cost proportional to use.

Contributions. (i) A serverless reference implementation, fully defined as infrastructure-as-code and covered by tests; (ii) an operational fleet simulator that produces realistic telemetry together with ground truth; (iii) a rule-aware ETA predictor with an ablation against a naive baseline; (iv) an order-independent, idempotent ingestion design validated by fault injection; (v) a web application with an operator control tower, a per-customer portal and public tracking links.

2 System design

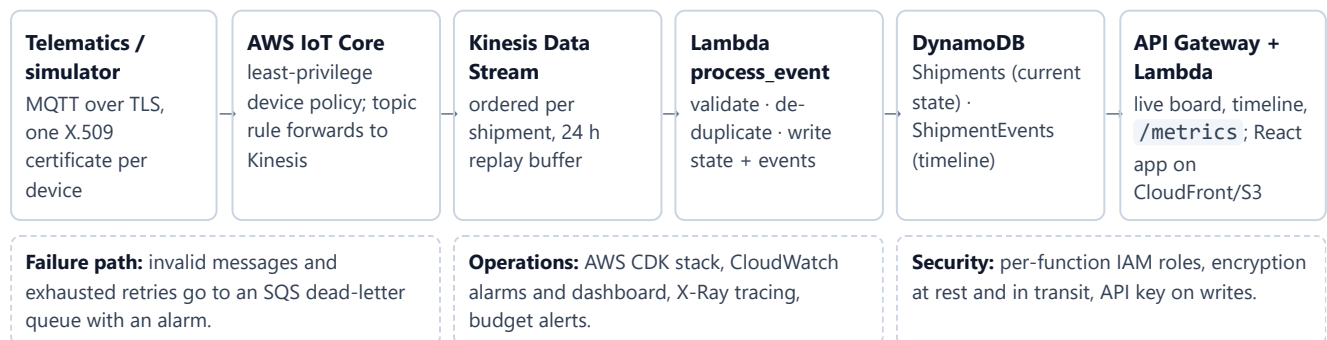


Figure 1. Architecture. Every component is managed and billed per use; the whole stack deploys with one `cdk deploy`.

Key design decisions.

Decision	Alternative	Rationale
Kinesis between IoT Core and Lambda	IoT rule invoking Lambda directly	Buffers bursts, keeps per-shipment order (partition key), and allows replay after a bug fix; costs one shard-hour.
Per-shipment sequence numbers + conditional writes	Last-writer-wins on timestamps	Makes processing idempotent and order-safe under at-least-once delivery without a separate de-duplication store (G2).
Two tables: current state + event log	Single state table	O(1) reads for the live board; one ordered Query for a shipment's timeline; delivered records drive the KPIs.
DynamoDB on-demand	PostgreSQL (RDS)	Access is by key, no joins; no idle cost, no VPC or patching (G4).
2 s REST polling	WebSocket push	Meets the 5 s target with far less state (no connection table); push is future work.
Partial-batch failure reporting	Fail whole batch	Only records after a failure are retried; poison messages go straight to the dead-letter queue.

Table 1. Design decisions and trade-offs.

Read model. The API derives everything a user sees from stored telemetry: predicted delay (ETA minus planned delivery), exceptions (late risk, temperature excursion, border hold, lost signal after 5 min, operator flag) and severity. Operator-owned fields (a manual flag) are written by a separate endpoint and merged on read, so the next GPS ping cannot overwrite them.

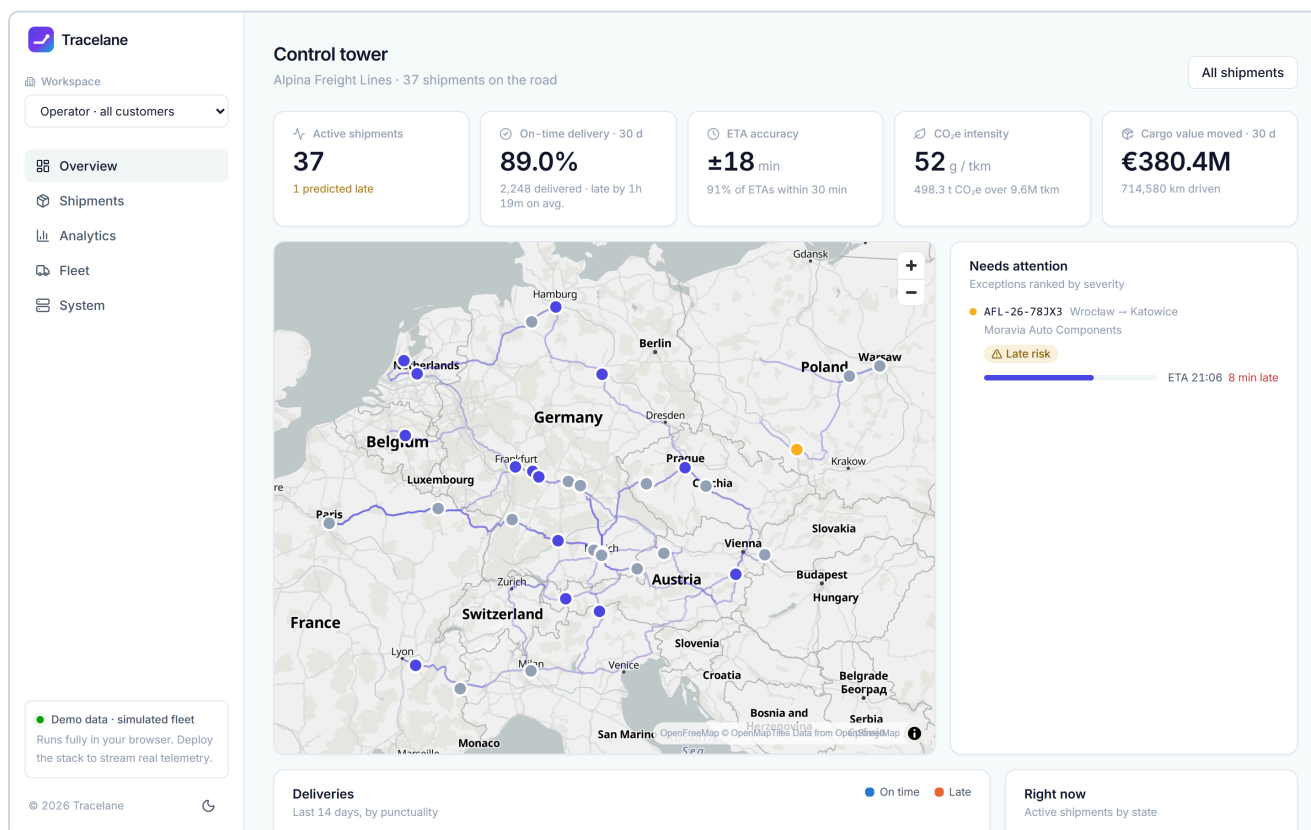


Figure 2. Operator control tower: live fleet on real road geometry, KPIs computed from recorded telemetry, and exceptions ranked by severity. Shippers see the same data restricted to their own shipments; consignees get a public tracking page.

3 Fleet simulation model

Real telematics data could not be obtained, and random-walk GPS points would make every metric meaningless. I therefore built a discrete-time simulator (1-minute steps) in which KPIs *emerge* from modelled operations. It is implemented once in Python (the device simulator) and ported to JavaScript for a zero-setup browser demo; both share one parameter file.

- **Network.** 156 directed lanes between 37 hubs (45–974 km) follow truck-drivable road geometry from OSRM on OpenStreetMap [4,5]; each lane is split into country segments (AT, BE, CH, CZ, DE, FR, HR, HU, IT, NL, PL, SI, SK) using Natural Earth boundaries [6].
- **Driving.** Cruise speed follows each country's truck speed limit plus limiter tolerance, urban speed near hubs, and random congestion that is more likely near cities at rush hour.
- **Driving-time rules.** A simplified form of Regulation (EC) 561/2006 [1]: a 45-minute break after 4.5 h of driving, an 11-hour rest after 9 h, and an extension to 10 h when it lets the driver finish the trip.
- **Borders.** Customs clearance whenever a lane enters or leaves Switzerland; random spot checks at Schengen internal borders.
- **Energy and emissions.** Consumption scales with payload; well-to-wheel factors for diesel, LNG, HVO and battery-electric trucks are approximated from the GLEC Framework [2] (cf. ISO 14083 [3]); tolls use indicative per-country rates.
- **Cargo.** 18 fictional shippers in 12 industries with realistic goods (HS tariff codes, ADR classes for dangerous goods), refrigerated loads whose units occasionally fail, and document sets that depend on route and cargo.
- **Planning.** When a load is assigned, a dispatcher plans the delivery time from expected drive time (scaled by a traffic factor), the driver's remaining hours, expected customs time, unloading and a customer-specific booking slack. On-time performance is therefore an outcome of planning policy versus stochastic events, not a random number.

The fleet has 40 trucks that chain loads: after delivery a truck takes its next job from the destination hub, so the network state evolves continuously. Aggregate outcomes (Table 2) are stable across seeds.

Outcome (30-day run)	mean $\pm$ sd over 5 seeds
Deliveries per 30-day run	2228 $\pm$ 42
On-time delivery rate	88.9 % $\pm$ 1.0 %
Mean lateness of late deliveries	83 min $\pm$ 11 min
Median transit time	7.0 h $\pm$ 0.2 h
Emission intensity, g CO ₂ e/tkm (well-to-wheel)	53.7 $\pm$ 0.6
Shipments with customs hold	7.9 % $\pm$ 1.0 %
Reefer loads with temperature excursion	3.9 % $\pm$ 2.0 %

Table 2. Operational outcomes of the simulated fleet.

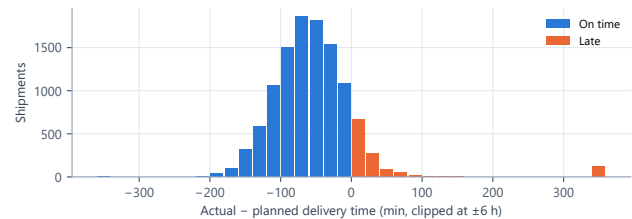


Figure 3. Actual minus planned delivery time, all seeds.

4 Evaluation

All experiments run from one script (`evaluation/run_experiments.py`) with fixed seeds; this report is generated from its output.

4.1 ETA prediction

For every delivered shipment I record the arrival time predicted at 10, 25, 50, 75 and 90 % of the route and compare it with the actual delivery time (n = 11,139 shipments per checkpoint). The baseline is the method in the original project proposal: remaining distance divided by a constant speed. Three variants add one model component at a time; the last one is the predictor the system uses.

Predictor	MAE @10 %	@25 %	@50 %	@75 %	@90 %	≤ 30 min @50 %	P90 @50 % (min)
Distance ÷ 80 km/h (baseline)	220	178	108	47	24	60 %	652
+ country speed profile	222	179	107	44	21	63 %	652
+ customs clearance	218	176	105	43	21	65 %	652
+ driving-time rules (deployed)	27	25	21	19	17	91 %	26

Table 3. Mean absolute ETA error in minutes by route progress, and accuracy at mid-route.

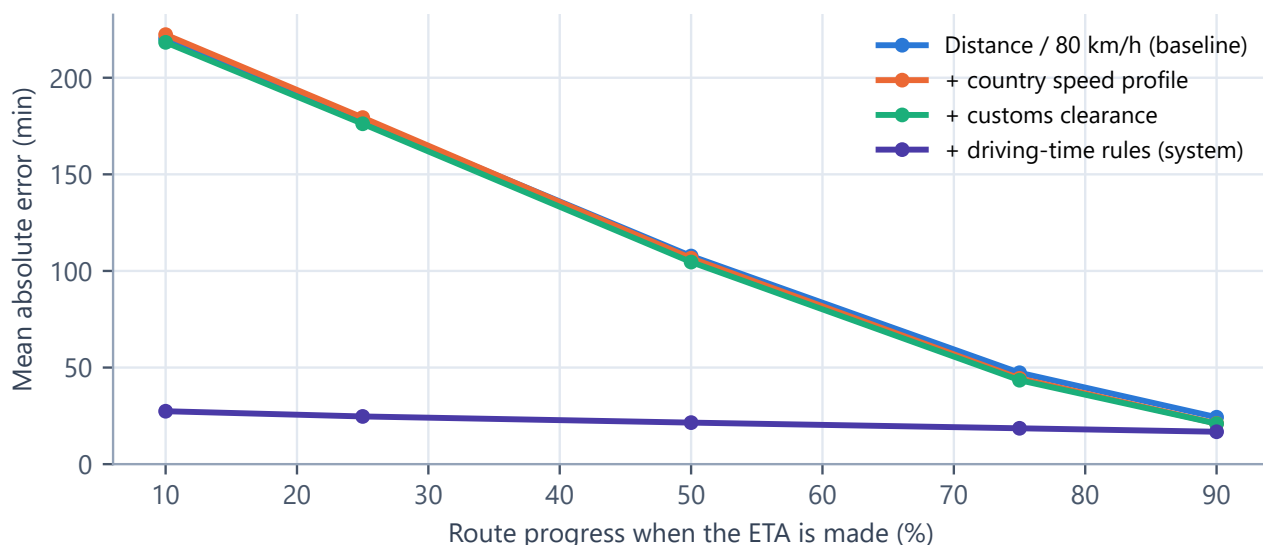


Figure 4. ETA error by route progress.

Encoding driving-time rules is what matters: speed profiles and customs alone barely change the error, because the dominant source of error on long lanes is whether a mandatory break or an 11-hour rest lies ahead. At 10 % progress the deployed predictor is within 30 minutes for 85 % of shipments (baseline: 46 %). Close to the destination the gap narrows (90 % progress: 17 vs. 24 min for the baseline), as little can happen on the last stretch.

The evaluation found a bug. A first run showed a cluster of predictions near the destination whose predicted arrival was about ten hours too *late*: when the daily driving limit and the 4.5-hour break limit fell close together, the predictor scheduled an 11-hour rest, while the driving model (correctly) let the driver take a 45-minute break and use the 10-hour extension. After rewriting the predictor to follow exactly the same decision order, 0.3 % of predictions at 90 % progress miss by more than two hours; I inspected these cases for one seed (7 of 2,170 predictions): all seven had a traffic jam after the prediction, which in six cases consumed the time the driver needed to finish within the extension and forced a daily rest the predictor could not foresee. **Caveat:** the predictor shares the simulator's rules, so this experiment measures the value of encoding operational constraints, not accuracy on real roads (Section 5).

4.2 Planning policy and on-time performance

Figure 5 varies the dispatcher's traffic factor and the booking slack. With the defaults the on-time rate is 89.3 % at a median early arrival of 61 min. Across the grid it ranges from 48 % to 98 %; buying the last points of on-time performance costs trucks waiting at the gate (median 166 min early at the most conservative setting). For a carrier the relevant quantity is therefore not the on-time rate alone but its trade-off with idle time — which the metrics API makes visible per customer and lane.

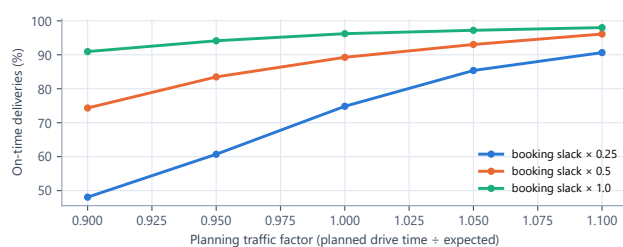


Figure 5. On-time rate vs. planning traffic factor for three booking-slack levels (one seed, 14 days).

4.3 Ingestion under faults

I fed the real `process_event` Lambda (against locally mocked DynamoDB and SQS) with 2,925 telemetry messages, once in order and once with 560 duplicates, 58 malformed messages and local reordering. Result: 302 of 302 shipments ended in an identical final state, the event log was identical (2,030 vs. 2,030 events), and all 58 malformed messages were routed to the dead-letter queue without retries. The first version of this experiment exposed a real bug: the "ETA at halfway" value behind the ETA-accuracy KPI was taken from whichever message *arrived* first. It is now written by a separate conditional update in which the lowest sequence number wins, making it independent of arrival order.

4.4 Throughput headroom

Messages average 1,757 bytes (P95 2,604). A Kinesis shard accepts 1 MB/s or 1,000 records/s [8]; with these sizes the byte limit binds first. At the realistic 30 s reporting interval one shard serves on the order of ten thousand trucks. Messages currently repeat static shipment data (parties, documents) in every ping; sending it once per shipment would cut message size by more than half.

Report interval	Trucks per shard	Msgs/day per 1,000 trucks
3 s	1,707	28.80 M
30 s	17,075	2.88 M
60 s	34,150	1.44 M

Table 4. Shard capacity from measured message sizes.

4.5 What was not measured

End-to-end latency (G1) is argued from the design — sub-second ingestion plus a 2 s poll — but was not measured on a deployed stack under load; the dashboard records per-message ingest latency (device timestamp → database), which a deployment would report. Cost (G4) follows AWS's pay-per-use model; the only fixed charge is the Kinesis shard.

5 Limitations and future work

- **Synthetic data.** The simulator is plausible, not validated: parameters were chosen to give credible aggregates, not fitted to real fleets. Because the ETA predictor encodes the same rules, Section 4.1 overstates real-world accuracy. The natural next step is learning ETA corrections (e.g. gradient-boosted residuals on lane, time of day and driver hours) from historical telemetry.
- **Tenancy.** The customer portal filters by customer ID; production use requires authentication (e.g. Cognito) that binds the customer ID to the caller's token, and unguessable tracking tokens.
- **Scale.** `/metrics` scans the state table, which is adequate for thousands of shipments; larger deployments need a secondary index and pre-aggregated daily metrics.
- **Push updates** via WebSockets, and a deployment-based latency and cost study, remain future work.

6 Conclusion

Tracelane shows that a small, fully serverless stack can deliver live freight visibility with correct semantics under at-least-once delivery, and that the most valuable "intelligence" for arrival prediction in road freight is not a better speed estimate but knowledge of the operational rules — breaks, rests and borders — that dominate long-haul timing. Building the simulator to evaluate this also turned out to be the most effective way to find correctness bugs in the pipeline.

References

1. Regulation (EC) No 561/2006 of the European Parliament and of the Council of 15 March 2006 on the harmonisation of certain social legislation relating to road transport. *Official Journal of the European Union*, L 102, 2006.
2. Smart Freight Centre. *Global Logistics Emissions Council Framework for Logistics Emissions Accounting and Reporting*, version 3.0, 2023.
3. ISO 14083:2023. *Greenhouse gases — Quantification and reporting of greenhouse gas emissions arising from transport chain operations*.
4. D. Luxen and C. Vetter. Real-time routing with OpenStreetMap data. In *Proc. 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2011.
5. OpenStreetMap contributors. OpenStreetMap, <https://www.openstreetmap.org> (data under ODbL).

6. Natural Earth. Admin 0 — Countries, 1:50m, <https://www.naturalearthdata.com>.
7. Amazon Web Services. *AWS Well-Architected Framework*, <https://docs.aws.amazon.com/wellarchitected/>.
8. Amazon Web Services. *Amazon Kinesis Data Streams Developer Guide* — quotas and limits.